

# NIFs and Ports — A Readable Companion to the OTP Docs

Chapter 11 · checked against OTP 29.1.1 · Sep 25, 2026

## 1. Why calling native code is a safety decision

Everything in the previous chapters rests on one guarantee: a failure in one process cannot corrupt another. Native code can break that guarantee. The moment C, C++, Rust or Zig runs inside the VM, a single bad pointer can take down every process on the node, and the supervision tree cannot help.

So choosing how to talk to native code is really choosing how much of that guarantee to give up in exchange for speed. The OTP docs are unusually direct about it: because a faulty NIF can cause problems “related to both stability and security it is recommended to use an external Port if possible.”

## 2. The building blocks

| Mechanism        | Runs where                                          | If the native code crashes                 | Cost per call                              | Use when                                                      |
|------------------|-----------------------------------------------------|--------------------------------------------|--------------------------------------------|---------------------------------------------------------------|
| Port             | Separate OS process, talks over stdin/stdout        | Only that OS process dies; the port closes | Highest: serialise and copy through a pipe | Default choice; existing tools and libraries                  |
| NIF              | Inside a VM scheduler thread                        | The whole VM crashes                       | Lowest: a function call                    | Small, hot, well-tested functions where port overhead matters |
| Linked-in driver | Inside the VM                                       | The whole VM crashes                       | Low                                        | Legacy; the docs recommend NIFs instead                       |
| C node           | Separate program speaking the distribution protocol | Only that program dies                     | Network round-trip                         | Native code that behaves like a peer node                     |
| Socket           | Any process, any machine                            | Only that program dies                     | Network round-trip                         | Services that already exist or live elsewhere                 |

In Elixir, Rustler (Rust) and Zigler (Zig) are the common ways to write NIFs. They remove many memory-safety bugs, but not the scheduling rules: a slow Rust NIF blocks a scheduler just like a slow C one.

## Ports in one paragraph

`open_port/2` starts an external program, and the process that opened it becomes its owner. Only the owner can talk to it. Data you send arrives on the program's stdin; what it writes to stdout arrives as `{Port, {data, D}}` messages. `{packet, N}` adds an N-byte length header so each message arrives whole. If the owner dies, the port closes, and a correctly written external program sees end-of-file on stdin and exits.

## NIFs in one paragraph

A NIF is a C function that replaces an Erlang function in a module. It receives terms in an *environment*, builds result terms in the same environment, and returns. It runs on the calling process's scheduler thread, is never preempted, and should return within about 1 ms or run on a dirty scheduler (chapter 10).

# 3. How the runtime actually does it

---

## Terms belong to environments

A term passed to a NIF is valid only inside the environment it came in, and only until the NIF returns. The docs say a term “is valid until its environment is destructed.” To keep data between calls, copy it into a process-independent environment (`enif_alloc_env` plus `enif_make_copy`) or keep it in native memory.

## Resource objects are how native memory meets the GC

To give Erlang a handle to native memory, allocate a resource (`enif_alloc_resource`), wrap it with `enif_make_resource`, and register a destructor. Erlang code passes the handle around like any term; when the garbage collector finds that nothing references it any more, the runtime calls your destructor. That's the only safe way to tie native lifetimes to Erlang lifetimes.

## NIFs run concurrently

The same NIF can run on every scheduler at once. It is thread-safe only if it “acts as a pure function and only reads the supplied arguments.” Any shared native state needs its own locking.

## Errors

`enif_make_badarg` makes the NIF raise `badarg` when it returns, whatever else it returns. Anything worse (a segfault, an abort, an out-of-bounds write) is not an exception: it is the end of the node.

## 4. Plausible but wrong

### 4.1 A NIF where a port would do

```
/* nif wrapping an image library to generate thumbnails */
static ERL_NIF_TERM thumbnail(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[]);
```

Image decoders parse untrusted input and have a long history of memory-safety bugs. Inside a NIF, one malformed upload crashes the whole node, and every connected user with it. Thumbnails take tens of milliseconds, so the port overhead is noise. Fix: run the tool as a port (or a pool of ports) under a supervisor; a crash then kills one OS process and restarts it.

### 4.2 Caching a term across calls

```
static ERL_NIF_TERM cached_config;

static ERL_NIF_TERM set_config(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[]) {
    cached_config = argv[0];          /* belongs to env */
    return enif_make_atom(env, "ok");
}
```

`argv[0]` dies with `env` when the NIF returns. Later calls read freed memory: sometimes the right value, sometimes garbage, sometimes a crash hours later with no obvious cause. Fix: copy it into an environment you own (`enif_make_copy` into an `enif_alloc_env` `env`), or convert it to a native struct.

### 4.3 Returning a raw pointer

```
return enif_make_uint64(env, (uint64_t)(uintptr_t) handle);
```

Erlang now holds an integer that looks like a handle. Nothing frees the memory when it's no longer used, a copy can be "closed" twice, and any integer can be passed back in and dereferenced. Fix: a resource object with a destructor; it is freed exactly once, when the garbage collector finds it unreferenced.

### 4.4 Shared state without locking

```
static int counter = 0;
static ERL_NIF_TERM next_id(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[]) {
    return enif_make_int(env, ++counter);
}
```

Calls from different schedulers run in parallel, so two callers can get the same id, which is a correctness bug that no Erlang test on a single core will reveal. Fix: an atomic increment or `enif_mutex_*`. Better still, keep the counter in Erlang (`ets:update_counter`, chapter 4) and keep the NIF pure.

## 4.5 A port whose program outlives its owner

```
P = open_port({spawn, "ffmpeg -i in.mp4 out.webm"}, [exit_status]).
```

If the owner crashes, the port closes and the external program's stdin reaches end-of-file. But `ffmpeg` here never reads stdin, so it keeps running, orphaned and invisible to your supervision tree. Repeat the crash and you have dozens of them. Fix: run tools through a small wrapper that exits when stdin closes, or use a process-management library that kills the child when its owner dies.

## 4.6 Printing debug output on the protocol channel

```
printf("got request of %d bytes\n", len); /* in a {packet, 2} port program */
```

stdout is the data channel. The debug line is read as a length header plus garbage, and the Erlang side gets corrupted messages or a hung protocol. The docs also warn that buffered stdio must not be used for the protocol itself. Fix: log to stderr, and write protocol bytes directly to file descriptor 1.

## 4.7 Building a shell command from input

```
convert(Filename) ->
  os:cmd("convert " ++ Filename ++ " out.png").
```

`os:cmd/1` runs its argument through a shell. A filename like `x.jpg`; `rm -rf ~` is a command, not a name. Fix: `open_port({spawn_executable, Path}, [{args, [Filename, "out.png"]}, exit_status])`. The arguments go straight to the program with no shell in between.

## 4.8 A stream port without framing

```
P = open_port({spawn_executable, Bin}, [binary]),
port_command(P, term_to_binary(Req)).
```

Without `{packet, N}` or `{line, L}`, the port is a byte stream: one message can arrive in pieces, and two can arrive glued together. The code works with small messages on an idle machine and fails with large ones under load. Fix: `{packet, 4}` on the Erlang side and matching length prefixes in the program, or `{line, MaxLen}` for text protocols.

# 5. Review checklist and sources

- ☐ Native code runs as a port unless measurements show the port overhead is unacceptable
- ☐ NIFs never keep terms from a process-bound environment after returning
- ☐ Native memory handed to Erlang is always a resource object with a destructor
- ☐ Shared native state is locked or atomic; ideally NIFs are pure
- ☐ NIFs finish within ~1 ms or are flagged dirty (chapter 10)
- ☐ Port programs exit on stdin end-of-file; orphaned programs are impossible
- ☐ Port programs log to stderr only

- No shell commands are built from input; `spawn_executable` with `args` is used instead
- Port protocols are framed with `{packet, N}` or `{line, L}`

## Sources

- Interoperability Overview: <https://www.erlang.org/doc/system/overview.html>
- Ports tutorial: [https://www.erlang.org/doc/system/c\\_port.html](https://www.erlang.org/doc/system/c_port.html)
- `erl_nif` reference: [https://www.erlang.org/doc/apps/erts/erl\\_nif.html](https://www.erlang.org/doc/apps/erts/erl_nif.html)
- `erlang:open_port/2` reference: <https://www.erlang.org/doc/apps/erts/erlang.html>